

ATIVIDADE 5: SIMULAÇÕES COM RTOS E MICROCONTROLADORES

Aluno: Leonardo Gabriel Santos Santana

Fase 1 — Fundamentos do ESP32

P01 — Pisca-pisca de LED interno

- 1. Que mudança no circuito ou no código gerou o efeito mais perceptível e por quê?** A alteração dos tempos na função `delay()` gerou o efeito mais perceptível, pois determinou diretamente o ritmo (frequência) e a simetria (tempo aceso versus tempo apagado) em que o LED interno pisca.
- 2. Como o ESP32 interpretou o sinal usado nesta prática: digital, analógico ou PWM?** O sinal foi interpretado como digital, utilizando níveis lógicos discretos (HIGH para ligado e LOW para desligado) através da atuação direta da função `digitalWrite()`.
- 3. Se você tivesse de tornar o experimento mais estável ou mais sensível, o que mudaria?** Para torná-lo mais estável e evitar o bloqueio total do processador entre os ciclos, substituiria a função `delay()` por uma lógica de temporização não bloqueante utilizando a função `millis()`.

P05 — Semáforo de três LEDs

- 1. Que mudança no circuito ou no código gerou o efeito mais perceptível e por quê?** A atribuição de tempos de espera de transição assimétricos (ex: muito tempo para o verde, pouco tempo para o amarelo) gerou o efeito mais notável, pois é o que de fato simula o comportamento real de um semáforo de trânsito.
- 2. Como o ESP32 interpretou o sinal usado nesta prática: digital, analógico ou PWM?** O ESP32 interpretou os sinais como puramente digitais, acionando as portas GPIO com `digitalWrite()` em níveis lógicos altos (HIGH) ou baixos (LOW) para comandar o acendimento dos LEDs.
- 3. Como você explicaria a lógica de estados deste experimento para outra equipe?** O sistema funciona como uma máquina de três estados finitos (vermelho, verde e amarelo) rodando em loop infinito. Em cada bloco, o LED do estado atual é acionado enquanto os outros são forçados ao estado apagado, aguardando um tempo predeterminado antes da transição obrigatória para a próxima cor da sequência.

P08 — Potenciômetro controla brilho por PWM

- 1. Que mudança no circuito ou no código gerou o efeito mais perceptível e por quê?** O giro mecânico do eixo do potenciômetro, pois alterou a tensão analógica recebida pela porta do microcontrolador, que reajustou em tempo real a largura de pulso (Duty Cycle) do PWM, provocando a mudança visível na intensidade luminosa do LED.
- 2. Como a variação do potenciômetro alterou os valores lidos e o comportamento final do sistema?** A rotação provocou uma divisão da tensão que o pino do ESP32 leu através do conversor analógico-digital (ADC) e gerou valores de 0 a 4095. O código traduziu essa escala para gerar a modulação PWM apropriada, controlando o brilho físico como resposta final.

3. Se você tivesse de tornar o experimento mais estável ou mais sensível, o que mudaria?

Para tornar o comportamento do brilho mais estável (sem "tremidas" por variação brusca), eu implementaria um filtro digital de média móvel nas leituras brutas do ADC para atenuar o ruído elétrico da protoboard.

P12 — Sensor de temperatura simulado por potenciômetro

1. Que mudança no circuito ou no código gerou o efeito mais perceptível e por quê? A

aplicação da função matemática de mapeamento gerou o efeito central da prática, pois ela pegou a resolução nativa do ADC (0 a 4095) e forçou a conversão proporcional para valores que nosso cérebro interpreta como graus Celsius, tornando os dados visíveis no console legíveis.

2. Como a variação do potenciômetro alterou os valores lidos e o comportamento final do sistema? Ao girar o componente, o hardware capturou diferentes valores de tensão e os converteu em um número decimal bruto. O software "fingiu" que aquele aumento de tensão era um aumento de temperatura de um sensor real, printando os novos valores escalonados via Serial.

3. Se você tivesse de tornar o experimento mais estável ou mais sensível, o que mudaria?

Para simular uma sensibilidade maior, restringiria o mapeamento dos limites na programação para representar uma janela de temperatura muito menor (ex: variando apenas de 35°C a 40°C em todo o giro do eixo), refletindo flutuações altíssimas para movimentos minúsculos.

P20 — Desafio integrador: estação básica de sinais

1. Que mudança no circuito ou no código gerou o efeito mais perceptível e por quê? A

coexistência e integração entre sensores analógicos e atuadores digitais dentro do mesmo script, mostrando na prática como uma variação de um pino de leitura pode definir o processamento que desvia o fluxo para ativar ou desativar portas de saída e geradores de PWM simultaneamente.

2. Como o ESP32 interpretou o sinal usado nesta prática: digital, analógico ou PWM? O

hardware processou as três grandezas: converteu níveis elétricos variáveis usando as portas analógicas (ADC), fez checagens lógicas discretas nas entradas digitais (botões) e exportou ondas retangulares para controle de potência por modulação PWM.

3. Se você tivesse de tornar o experimento mais estável ou mais sensível, o que mudaria?

Faria melhorias diretas por software, aplicando debounce rigoroso nos acionamentos dos botões (para impedir cliques falsos) e reduzindo a taxa de amostragem de `Serial.println()` para não encher o buffer, garantindo que o tempo do microcontrolador seja focado na reação.

Fase 2 — Conceitos básicos de tempo real

P21 — Pisca-pisca sem delay com millis

1. Qual ajuste temporal mais alterou a previsibilidade do sistema e como isso foi

percebido? A remoção completa do `delay()` e uso do delta temporal via `millis()` alterou de vez o comportamento. Percebe-se isso notando que o ESP32 agora verifica continuamente o tempo, sem jamais paralisar sua capacidade de processar outras informações no ciclo de instrução.

2. Em que situação o uso de delay() pioraria esta prática em relação à solução atual? Se tivéssemos de monitorar a queda brusca de uma porta de emergência (botão) na mesma rotina, o delay() faria o processador perder totalmente o evento se este ocorresse exatamente durante o período congelado de pausa do código.

3. Que evidência experimental o grupo registrou para justificar sua conclusão? Ao inserir comandos de contagem livre fora da lógica de tempo do LED no loop(), registrou-se no monitor que o microcontrolador contabilizou milhares de ciclos perfeitamente paralelos ao momento em que o pisca-pisca aguardava sua vez de alternar de estado.

P25 — Debounce temporal de botão

1. Como o sistema reagiu ao pressionamento rápido ou repetido do botão? Graças ao bloqueio da histerese temporal configurada (a janela de tempo cega), o ESP32 ignorou intencionalmente os ruídos mecânicos das chapas metálicas do contato e não registrou dezenas de "toques fantasmas", contabilizando apenas os acionamentos humanos válidos.

2. Em que situação o uso de delay() pioraria esta prática em relação à solução atual? Aplicar delay() de propósito sempre que o botão for tocado paralisaria inteiramente todos os sensores da placa a cada ação manual humana. Se existisse um atuador que necessitasse de cálculos complexos em tempo hábil, ele sofreria desvios temporais catastróficos.

3. Que evidência experimental o grupo registrou para justificar sua conclusão? A evidência foi testar e alterar a janela de milissegundos para exclusão (ex: aumentar de 10ms para 150ms). Os registros demonstraram que a anomalia de um clique gerar múltiplas saídas seriais desapareceu ao impor a barreira correta baseada no tempo do millis().

P32 — Sequenciador de estados com tempos configuráveis

1. Qual ajuste temporal mais alterou a previsibilidade do sistema e como isso foi percebido? A capacidade de atualizar constantemente as variáveis alvo com os retornos do millis() a depender da transição da máquina de estados permitiu moldar fases velozes e morosas sem interromper o desempenho do laço base do C++.

2. Em que situação o uso de delay() pioraria esta prática em relação à solução atual? A imobilização criada pelo atraso via hard code impossibilitaria qualquer manobra evasiva ou escalonamento cooperativo caso quiséssemos integrar o sequenciador com uma leitura analógica ou comunicação Wi-Fi concorrente na mesma malha.

3. Que evidência experimental o grupo registrou para justificar sua conclusão? A leitura serial contínua comprovou matematicamente que o microcontrolador prosseguiu suas rotinas sem interrupções e apenas realizou o chaveamento na fronteira exata da transição em que o relógio absoluto superou o cronômetro do estado ativo.

P35 — Tarefa rápida e tarefa lenta compartilhando loop

1. Qual ajuste temporal mais alterou a previsibilidade do sistema e como isso foi

percebido? O isolamento de duas tarefas em blocos de checagem temporal individual. Isso demonstrou o conceito puro de divisão de tempo (Time Slicing cooperativo), onde a tarefa rápida intercala execuções múltiplas enquanto a lenta acumula seu tempo nos bastidores de forma previsível.

2. Em que situação o uso de delay() pioraria esta prática em relação à solução atual? Se as pausas fossem estruturadas com delay, a tarefa rápida estaria perpetuamente refém da cadência letárgica do código da tarefa lenta, destruindo totalmente as taxas de amostragem independentes exigidas pela atividade.

3. Que evidência experimental o grupo registrou para justificar sua conclusão? O painel evidenciou fisicamente as pisadas em ritmos díspares, onde uma via completava rotinas com alta frequência enquanto a segunda se mantinha dormiente até ser reativada estritamente pela expiração de seu período longo.

P40 — Desafio integrador: sistema reativo sem RTOS

1. Qual ajuste temporal mais alterou a previsibilidade do sistema e como isso foi

percebido? A sincronização harmônica entre as diferentes lógicas de millis(). O escalonamento puramente cooperativo funcionou como um maestro, onde o sistema se manteve globalmente reativo perante interações analógicas e físicas sem que um "evento engolisse o outro".

2. Em que situação o uso de delay() pioraria esta prática em relação à solução atual? Neste desafio integrador, qualquer congelamento injetaria "jitter" (desvio da resposta esperada) nas frequências programadas, corrompendo atuações de PWM e gerando falhas para interceptar estímulos de curtíssima duração.

3. Que evidência experimental o grupo registrou para justificar sua conclusão? O grupo registrou experimentalmente que a inserção de apenas um laço moroso nas funções impactou negativamente toda a topologia. Ao remover bloqueios, toda a estação passou a despachar requisições assíncronas perfeitamente alinhadas nos prazos simulados.

Fase 3 — RTOS no ESP32

P41 — Primeira task FreeRTOS piscando LED

1. Como a divisão em tasks ajudou ou atrapalhou a compreensão do comportamento do

sistema? A divisão modularizou profundamente a lógica, o que ajudou na legibilidade do software. Transferimos o processo de atuar o LED do loop monolítico para um processo confinado e delegado inteiramente para o escalonador preemptivo nativo do ESP32.

2. Qual alteração teve maior impacto: prioridade, sincronização, fila ou tempo de espera? O tempo de espera via vTaskDelay() teve o maior impacto funcional, pois ele não perde recursos. Ao entrar nesse estado, a task entrega explicitamente o domínio do núcleo da CPU para outras tarefas menores (ou para o modo Idle) de forma arquitetural.

3. Que cuidado de projeto você adotaria se esta prática fosse levada para um sistema real?

Em um produto final de engenharia, haveria o rigor absoluto de calcular empiricamente e prever a

quantidade de memória (Stack Size) alocada pela função `xTaskCreate` a fim de impossibilitar invasão do montante e estouros crônicos por *Stack Overflow*.

P45 — Fila simples entre sensor e atuador

- 1. Como a divisão em tasks ajudou ou atrapalhou a compreensão do comportamento do sistema?** Ajudou ao expor claramente o paradigma arquitetural Produtor/Consumidor. O sensor não conhece o atuador, sua responsabilidade se restringe unicamente a despachar leituras cruas, permitindo que as rotinas rodem de forma descentralizada.
- 2. Qual alteração teve maior impacto: prioridade, sincronização, fila ou tempo de espera?** A implementação da Fila (Queue) intertarefas. Ela serviu como via diplomática blindada para os dados da memória transacionarem sem que variáveis de acesso compartilhado bruto entrassem em choque ou causassem concorrência de leitura errática.
- 3. O sistema ficou mais fluido com fila maior ou menor? O que seus registros indicam?** Os registros indicam que ao superdimensionar o vetor, a fluidez se manteve com um custo grande na memória base, já no caso inverso, picos de leituras produtivas causaram descartes imprevistos de dados por estrangulamento da caixa postal minúscula.

P48 — Mutex para proteger Serial

- 1. Como a divisão em tasks ajudou ou atrapalhou a compreensão do comportamento do sistema?** Na ausência de controle, as tarefas assíncronas concorrentes corrompiam propositalmente a visualização por atropelarem a interface Serial em paralelo. A confusão revelou o porquê de processos necessitarem de uma governança protetiva restritiva.
- 2. Qual alteração teve maior impacto: prioridade, sincronização, fila ou tempo de espera?** A sincronização via inclusão de semáforos de exclusão mútua (Mutex). Ao obrigar a tarefa a pegar a "chave" antes de redigir log, as colisões de formatação sumiram instantaneamente, e o acesso à porta limitou-se ao respectivo detentor atual.
- 3. Que cuidado de projeto você adotaria se esta prática fosse levada para um sistema real?** Implantaria táticas para contornar inversão de prioridade em caso de contenção extrema e asseguraria que os blocos entre as invocações de Tomar e Devolver o sinalizador no código fossem absurdamente rápidos, evitando *Deadlocks* do Kernel.

P52 — Software timer aciona LED

- 1. Como a divisão em tasks ajudou ou atrapalhou a compreensão do comportamento do sistema?** Ajudou fortemente o uso inteligente de APIs. Diferente de levantar uma Thread base estática apenas para "contar", chamar uma rotina atrelada ao servidor de temporizadores nativos do FreeRTOS reduziu severamente o dispêndio sistêmico.
- 2. Qual alteração teve maior impacto: prioridade, sincronização, fila ou tempo de espera?** O disparo temporal preciso. O hardware foi capaz de deflagrar funções limpas na sub-rotina de Callback exatamente no período solicitado, sendo processado pela tarefa Daemon nativa (Timers) do modo oculto no fundo.

3. Que cuidado de projeto você adotaria se esta prática fosse levada para um sistema real?

Aplicaria regras drásticas de segurança no Callback: proibiria loops matemáticos longos, não permitiria que a rotina invocasse métodos `vTaskDelay`, garantindo de forma implacável a execução quase atômica para não asfixiar os outros despachos na fila do sistema de tempo.

P60 — Desafio integrador: mini sistema RTOS

1. Como a divisão em tasks ajudou ou atrapalhou a compreensão do comportamento do sistema?

Consolidou a compreensão arquitetural avançada ao fatiar componentes de atuação paralelos (Leituras base, relatórios e sinais visuais) como blocos corporativos em que cada tarefa opera uma peça do xadrez, conferindo tolerância a falhas na camada embarcada.

2. Qual alteração teve maior impacto: prioridade, sincronização, fila ou tempo de espera?

O balanceamento orquestrado geral (preempção por prioridade cruzada à sincronização). Garantir o escalonamento em que o fluxo crítico intercepte a fila e usurpe temporariamente os núcleos processuais com eficácia provou as bases rigorosas do Tempo Real dinâmico.

3. Que cuidado de projeto você adotaria se esta prática fosse levada para um sistema real?

Auditorias restritas de código usando rastreadores RTOS (Trace), asseguraria a injeção contínua de cães-de-guarda lógicos via Task Watchdog (TWDT) em cada laço essencial para reinicializar o sistema por *Hard Fault* em instâncias não responsivas críticas.